NOTES

CS445 — Web Programming
IIIT Vadodara Campus Event Board

ASSIGNMENT 6 — QUESTION-WISE ANSWERS

Team leader- Student Name: Payal verma

Roll Number: 20261651058

Team members-

Student Name: Piyush jain

Roll Number: 20261651060

Student Name: Prince vijay

Roll Number: 20261651066

Section A — JavaScript Fundamentals

A1. Declare variables using const and let

const is used when a variable should not be reassigned. let is used when its value may change. Both are block-scoped.

```
const boardTitle = "IIIT Vadodara Event Board";
let totalEvents = 0;
```

A2. JavaScript data types

The project uses string, number, and boolean data types. A string stores text, a number stores numeric values, and a boolean stores true or false.

```
const title = "Web Development Workshop"; // string
const seats = 30;                        // number
const online = true;                       // boolean
```

A3. Create an array of event objects

An object stores related properties of one event. An array stores multiple event objects.

```
const events = [
  { title: "Web Development Workshop", date: "2026-11-10", seats: 30, online: true },
  { title: "Cyber Security Seminar", date: "2026-11-12", seats: 25, online: false }
];
```

A4. Use if/else to display event labels

The condition checks seats and returns Full, Few left, or Open.

```
function getStatus(seats) {
  if (seats === 0) return "Full";
  else if (seats < 10) return "Few left";
  else return "Open";
}
```

A5. Function with parameter and return value

A function receives a parameter, performs an operation, and returns a result to the caller.

```
function formatEvent(event) {
  return event.title + " - " + event.date;
}
```

A6. Arrow function and array method

An arrow function is a short function syntax. filter() creates a new array containing only elements that pass the condition.

```
const availableEvents = events.filter(event => event.seats > 0);
```

A7. Process events using a loop

A for...of loop visits each item of an array one by one.

```
for (const event of events) {
  console.log(event.title);
}
```

A8. Explain variable scope

Scope is the area of a program where a variable can be accessed. A variable declared inside a function is local to that function. let and const are block-scoped. A variable declared outside a function can be accessed in the wider script scope, subject to its declaration.

Section B — DOM, Events, Fetch and Validation

B1. Select elements from the DOM

DOM means Document Object Model. It represents an HTML document as a tree of objects. querySelector() selects the first matching CSS selector, querySelectorAll() selects all matching elements, and getElementById() selects an element by its ID.

```
const eventContainer = document.querySelector("#events");
const searchInput = document.querySelector("#event-search");
const menuButton = document.getElementById("menu-toggle");
```

B2. Create and render event cards

createElement() creates an element, textContent adds text safely, and appendChild() inserts it into a parent.

```
const card = document.createElement("article");
card.className = "event-card";
card.textContent = event.title;
eventContainer.appendChild(card);
```

B3. Update content, class, style and attributes

textContent changes text; classList adds/removes CSS classes; style changes inline CSS; setAttribute() adds or updates an HTML attribute.

```
card.textContent = event.title;
card.classList.add("event-card");
card.style.border = "1px solid gray";
card.setAttribute("data-event-id", event.id);
```

B4. Toggle the hamburger menu

addEventListener() registers a click handler. classList.toggle() adds the class if absent or removes it if present.

```
menuButton.addEventListener("click", () => {
  nav.classList.toggle("open");
});
```

B5. Explain event.target and event bubbling

event.target is the element where the event started. Event bubbling is the movement of an event from its target up through parent elements. Event delegation uses bubbling by placing one listener on a parent and checking the target.

B6. Search events and handle Escape

The input event runs when the user types in the search box. The events can be filtered by title. The Escape key can clear the input and show all events again.

```
searchInput.addEventListener("input", () => {
  const query = searchInput.value.toLowerCase();
```

```
  const filtered = events.filter(event => event.title.toLowerCase().includes(query));
  renderEvents(filtered);
});
searchInput.addEventListener("keydown", event => {
  if (event.key === "Escape") {
    searchInput.value = "";
    renderEvents(events);
  }
});
```

B7. Fetch JSON with loading, success and error states

fetch() requests a file from a server. response.ok checks HTTP success. response.json() parses JSON. try/catch handles errors and lets the page show a friendly message.

```
async function loadJSON() {
  eventContainer.textContent = "Loading events...";
  try {
    const response = await fetch("events.json");
    if (!response.ok) throw new Error("JSON file not found");
    const events = await response.json();
    renderEvents(events); // success
  } catch (error) {
    eventContainer.textContent = "Unable to load events. Please try again.";
    console.error(error);
  }
}
```

B8. Validate a form

required makes a field compulsory. type=email checks email format. pattern checks a value against a regular expression. setCustomValidity() sets a custom error. checkValidity() checks the form and preventDefault() prevents invalid submission.

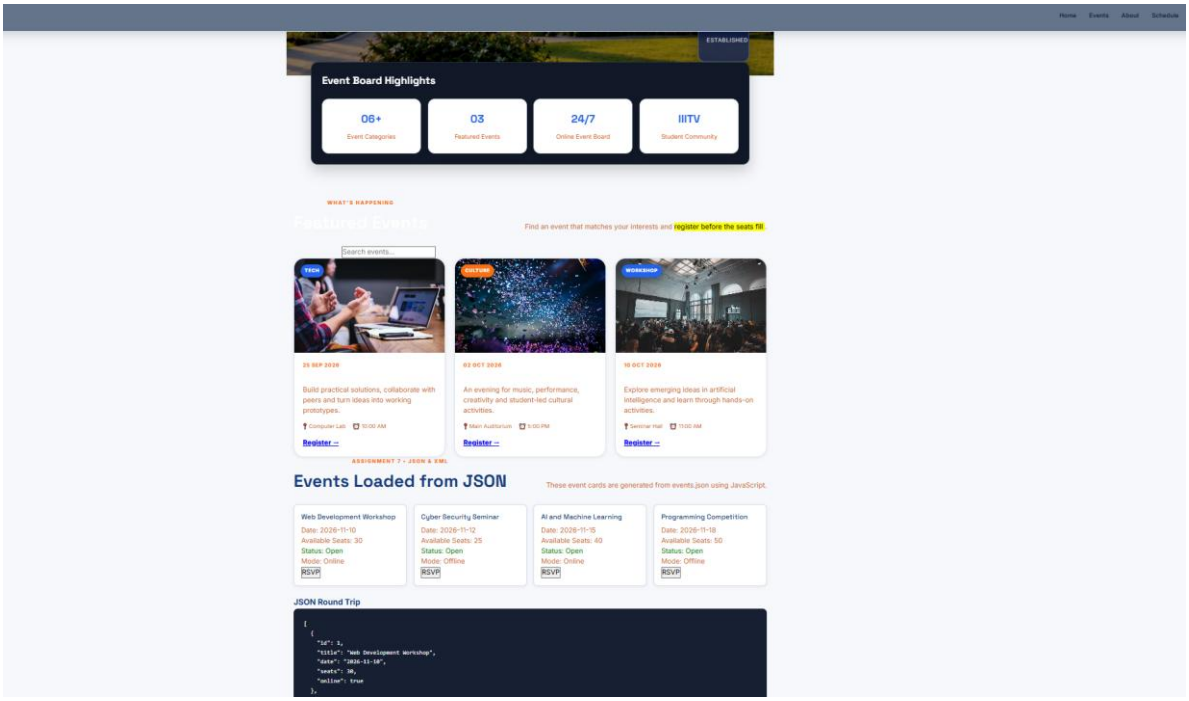
```
emailInput.setCustomValidity(
  emailInput.value.endsWith("@iiitvadodara.ac.in") ? "" : "Use your institute email."
);
if (!form.checkValidity()) {
  event.preventDefault();
  form.reportValidity();
}
```

Client-side validation provides quick feedback, but server-side validation is also required because users can bypass browser checks.

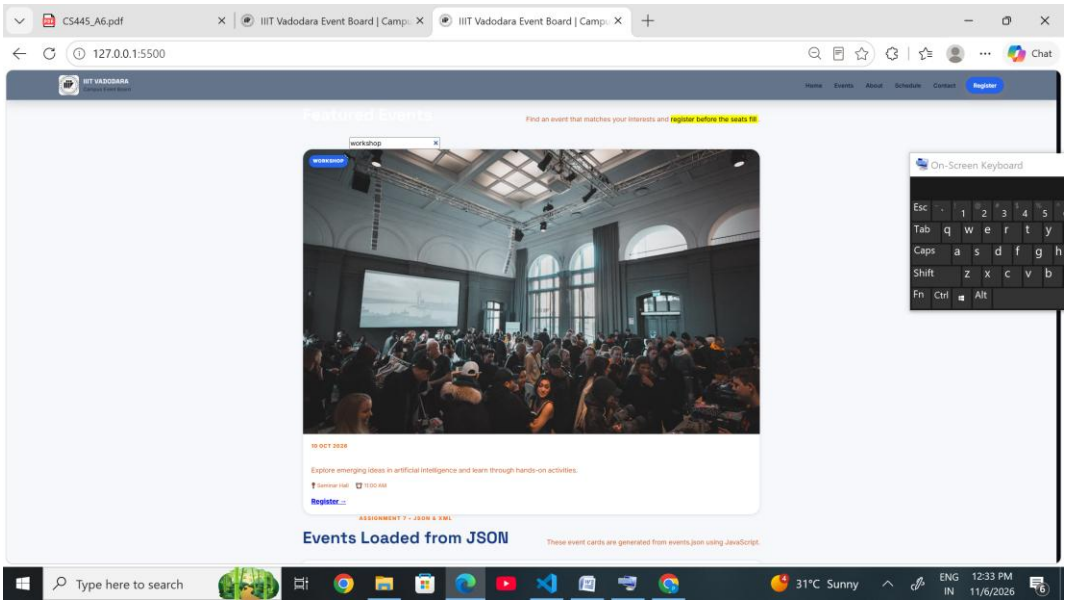
B9. Use browser Developer Tools

The Console tab displays logs and JavaScript errors. The Network tab helps inspect Fetch requests and their status. Screenshots should show the running event board, search, Fetch states, and form validation.

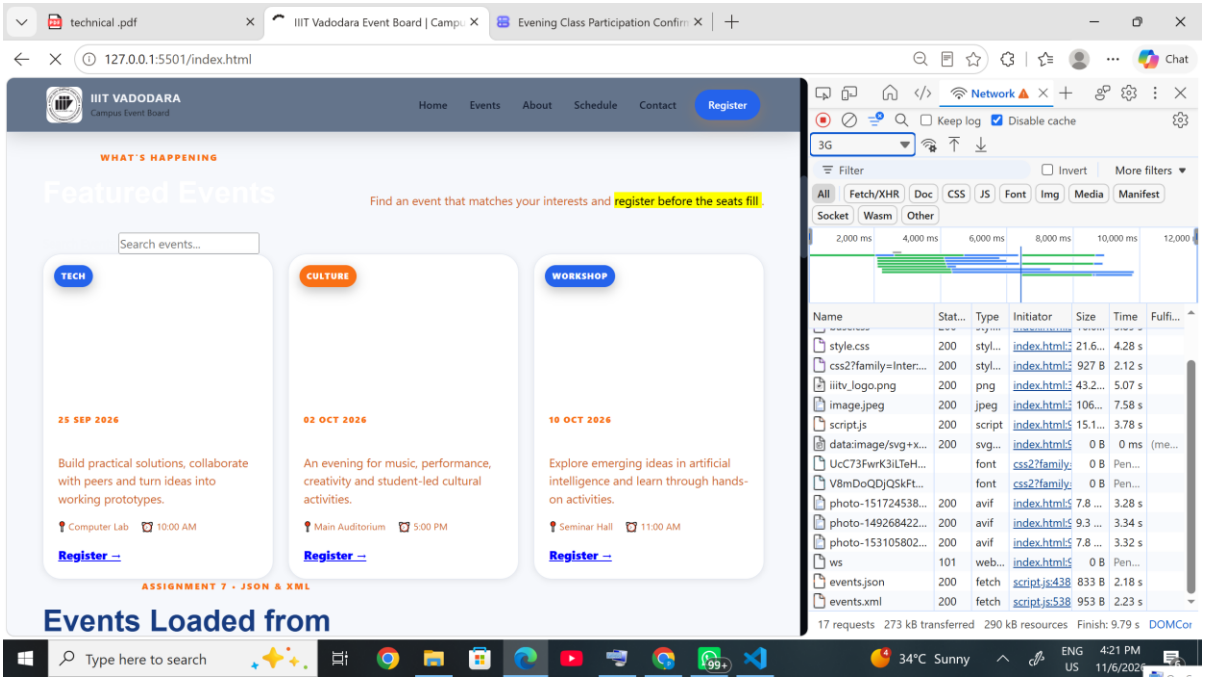
Events rendered from data



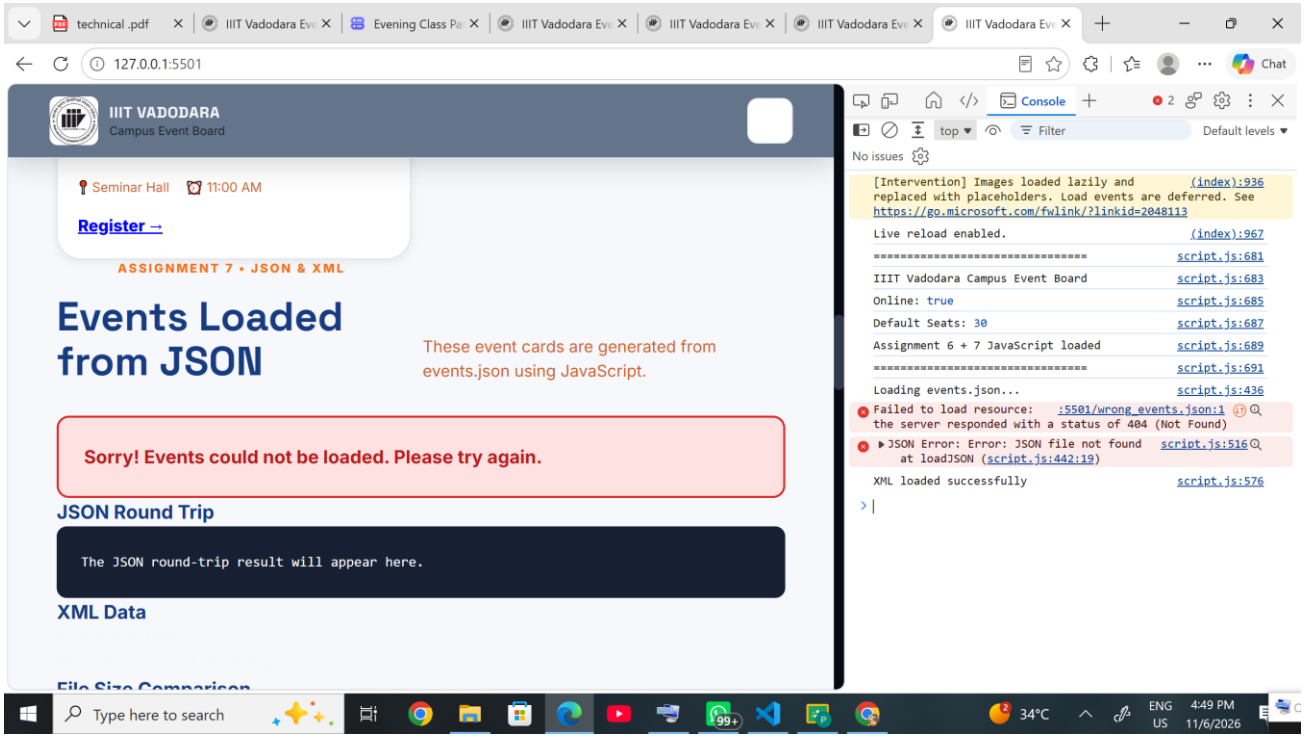
Search filter working



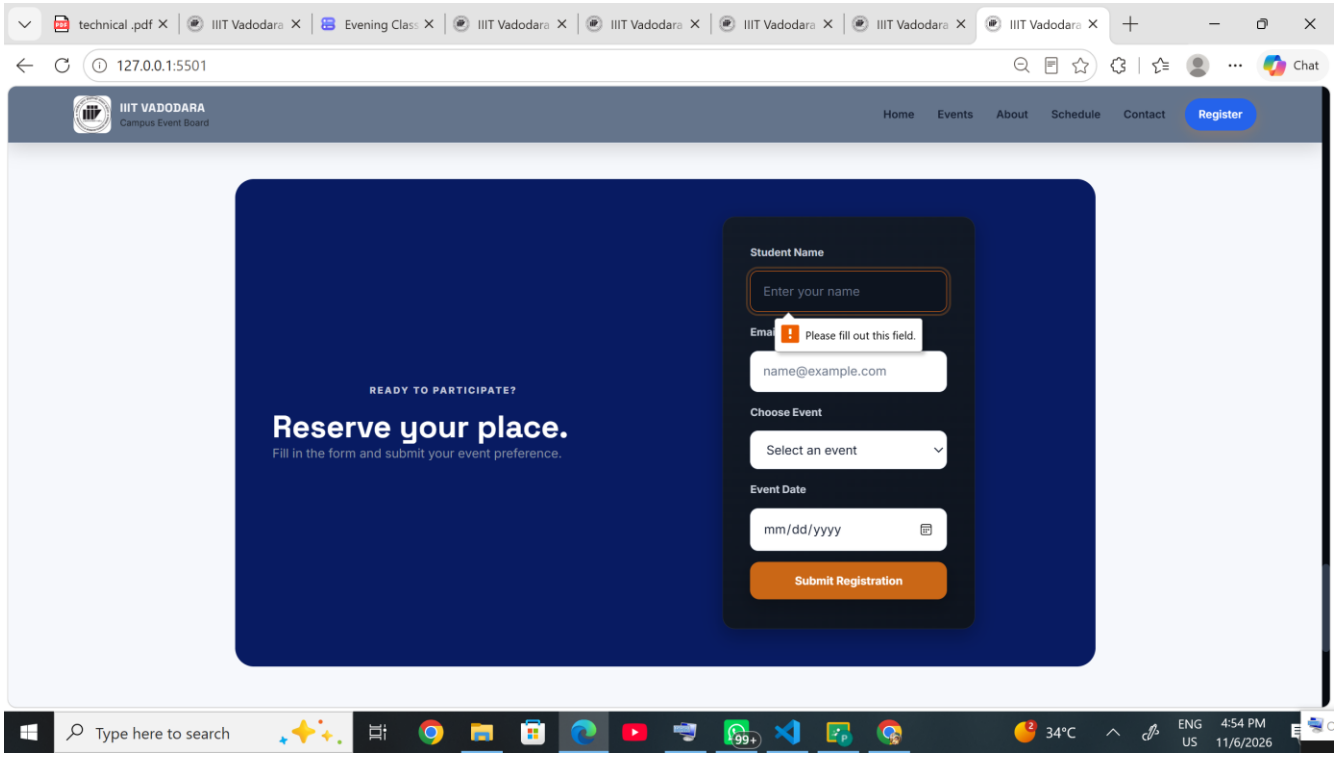
Fetch Success state and loading events :-



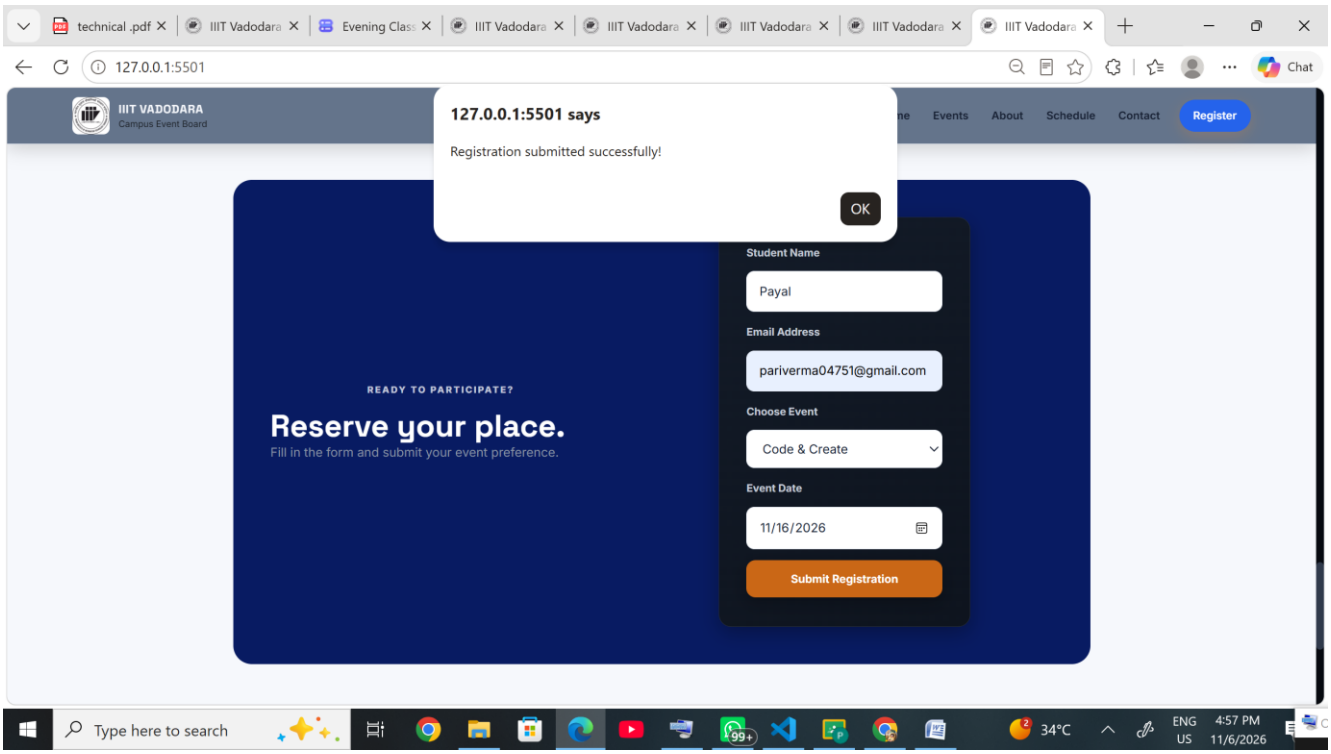
05_Fetch_Error



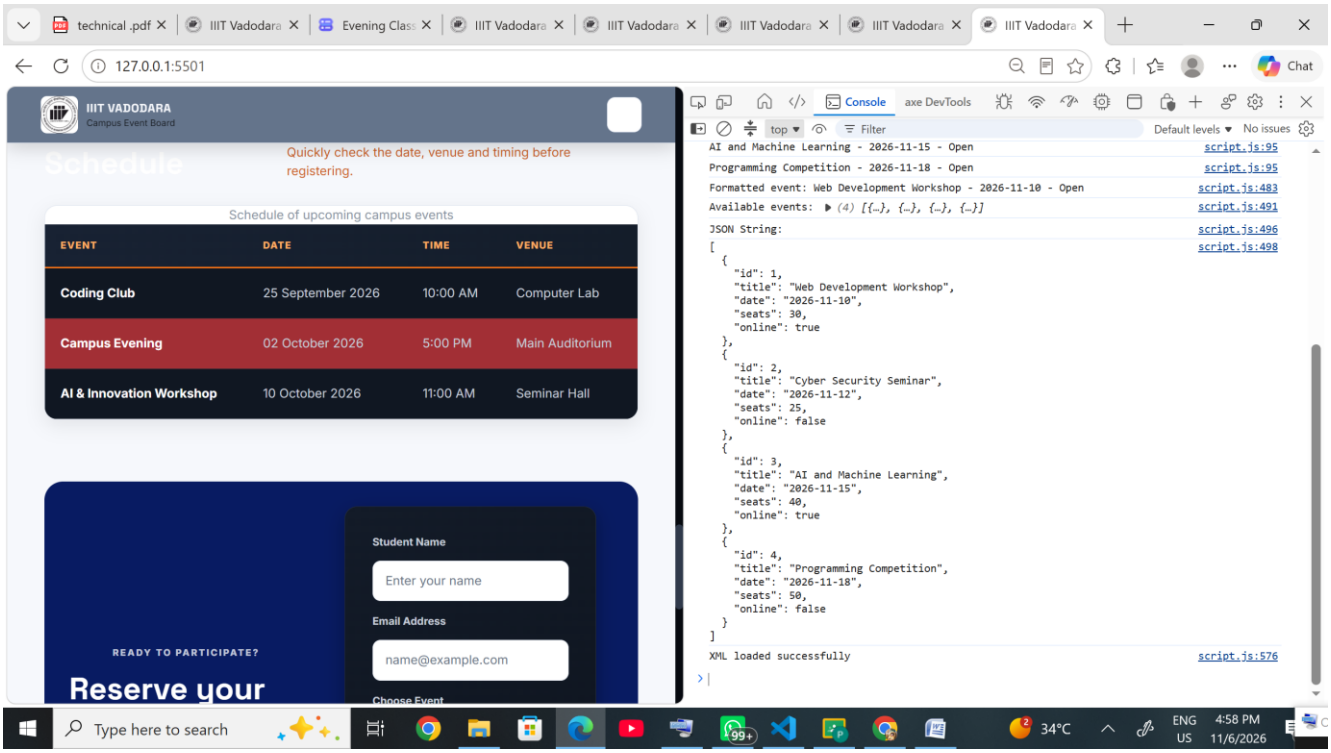
06_Form_Validation_Error.



07_Form_Success.png



08_Console_Output



Assignment 6:- Ques –ans

1. let vs const — when do you use each, and why avoid var?

`const` is used when a variable should not be reassigned.

`let` is used when a variable's value may change. Both are block-scoped.

We avoid `var` because it is function-scoped rather than block-scoped, which can cause unexpected behavior and variable conflicts.

```
const boardTitle = "IIIT Vadodara Event Board";
let totalEvents = 0;
```

2. List the data types you used and give one value of each.

The project uses the following JavaScript data types:

- **String:** `"Web Development Workshop"`
- **Number:** `30`
- **Boolean:** `true`

Examples:

```
const title = "Web Development Workshop";
const seats = 30;
const online = true;
```

3. Show one event object from your array. Why is an array of objects a good model for this page?

Example of an event object:

```
{
    title: "Web Development Workshop",
    date: "2026-11-10",
    seats: 30,
    online: true
}
```

An array of objects is a good model because each object represents one event with related properties. The array stores multiple events together, making it easy to display, search, filter, and manage events.

4. Paste your control-flow rule from A4. What labels did it produce for your events?

The following `if/else` rule checks the available seats:

```
function getStatus(seats) {
    if (seats === 0)
        return "Full";
    else if (seats < 10)
        return "Few left";
    else
        return "Open";
}
```

For the two sample events in my notes:

- Web Development Workshop — 30 seats: **Open**
- Cyber Security Seminar — 25 seats: **Open**

5. Show one function and one arrow function with an array method. What does each return?

Normal function:

```
function formatEvent(event) {
    return event.title + " - " + event.date;
}
```

It returns a formatted string containing the event title and date.

Arrow function with an array method:

```
const availableEvents = events.filter(event => event.seats > 0);
```

The arrow function checks whether an event has more than zero seats. The `filter()` method returns a new array containing only available events.

6. Which elements did you select, and with which methods? Why prefer querySelector?

I selected the following DOM elements:

```
const eventContainer = document.querySelector("#events");
const searchInput = document.querySelector("#event-search");
const menuButton = document.getElementById("menu-toggle");
```

`querySelector()` is preferred because it supports CSS selectors such as IDs, classes, and element names. It selects the first matching element.

7. Explain event bubbling in one or two lines, using event.target from B5.

Event bubbling means an event starts at the target element and then moves upward through its parent elements.

`event.target` identifies the element where the event originally occurred. Event delegation uses this behavior to handle events through a parent element.

8. Describe the three states your fetch handles, and how the user sees each.

My JSON Fetch handles three states:

1. **Loading:** The page displays "Loading events..." while data is being fetched.
2. **Success:** When the JSON file loads successfully, the event data is displayed as event cards.
3. **Error:** If fetching fails, the page displays "Sorry! Events could not be loaded. Please try again."

The `try...catch` block handles errors, and `response.ok` checks whether the HTTP request was successful.

9. Show your built-in validation attributes and your custom setCustomValidity rule. Why must the server validate again?

Built-in HTML validation attributes used in the form include:

- `required` — makes a field compulsory.
- `type="email"` — checks the email format.
- `pattern` — checks a value against a regular expression.

Custom validation rule:

```
emailInput.setCustomValidity(
    emailInput.value.endsWith("@iiitvadodara.ac.in")
        ? ""
        : "Use your institute email."
);
```

The form is checked using:

```
if (!form.checkValidity()) {
    event.preventDefault();
    form.reportValidity();
}
```

Server-side validation is also necessary because users can bypass client-side validation. The server must independently check the submitted data for security and correctness.

10. Give the GitHub repo link (and Pages link if live); confirm a README and readable commit history; paste your final script.js.

GitHub Repository: https://github.com/jannatverma/event-board

Final script.js:

```
/* ================================================
   IIIT VADODARA CAMPUS EVENT BOARD
   CS445 - ASSIGNMENT 6 + ASSIGNMENT 7
   ================================================ */


/* ================ A1 + A2: VARIABLES ================ */

// String
const boardTitle = "IIIT Vadodara Campus Event Board";

// Number
const defaultSeats = 30;

// Boolean
const isOnline = true;

// let: value can change
let totalEvents = 0;

// A3: Array of event objects (loaded from JSON)
let events = [];


/* ================ B1: SELECT ELEMENTS ================ */

const container = document.getElementById("json-events");

const jsonOutput = document.querySelector("#json-output");

const xmlTitle = document.querySelector("#xml-title");

const fileSize = document.querySelector("#file-size");

const menuButton = document.querySelector(".menu-toggle");

const navLinks = document.querySelector(".nav-links");

const eventForm = document.querySelector(".event-form");

const studentName = document.querySelector("#student-name");

const searchBox = document.querySelector("#event-search");

const navItems = document.querySelectorAll(".nav-links a");
```

```
/* ================ A4: IF / ELSE ================ */

// Decide event availability
function getStatus(event) {

    if (event.seats === 0) {
        return "Full";
    }

    else if (event.seats < 5) {
        return "Few left";
    }

    else {
        return "Open";
    }

}



/* ================ A6: FUNCTION + RETURN ================ */

// Function with parameter and return value
function formatEvent(event) {

    return `${event.title} - ${event.date} - ${getStatus(event)}`;

}



/* ================ A7: ARROW FUNCTION ================ */

// Arrow function + filter()
const availableEvents = () => {

    return events.filter(event => event.seats > 0);

};



/* ================ A5: FOR...OF LOOP ================ */

function processEvents() {

    for (const event of events) {

        console.log(formatEvent(event));

    }

}



/* ================ B2 + B3: RENDER JSON EVENT CARDS ================ */

function renderEvents(eventList) {

    if (!container) {
        console.error("Events container not found");
        return;
    }

    // Clear previous cards
    container.innerHTML = "";

    // Loop through events
    for (const event of eventList) {

        // Create event card
        const card = document.createElement("article");

        card.classList.add("event-card");

        // Event title
        const title = document.createElement("h3");

        title.textContent = event.title;

        // Event date
        const date = document.createElement("p");

        date.textContent = "Date: " + event.date;

        // Available seats
        const seats = document.createElement("p");

        seats.textContent = "Available Seats: " + event.seats;

        // Event status
        const status = document.createElement("p");
```

```
        status.textContent = "Status: " + getStatus(event);

        // Change classes and styles
        if (event.seats === 0) {

            card.classList.add("full");

            status.style.color = "red";

        }

        else if (event.seats < 5) {

            card.classList.add("few-left");

            status.style.color = "orange";

        }

        else {

            card.classList.add("open");

            status.style.color = "green";

        }

        // Add custom attribute
        card.setAttribute("data-event-title", event.title);

        // Online / Offline information
        const online = document.createElement("p");

        online.textContent = event.online
            ? "Mode: Online"
            : "Mode: Offline";

        // Create RSVP button
        const button = document.createElement("button");

        button.type = "button";

        button.classList.add("rsvp-button");

        button.textContent = event.seats === 0
            ? "Event Full"
            : "RSVP";

        button.disabled = event.seats === 0;

        // Append elements to card
        card.appendChild(title);

        card.appendChild(date);

        card.appendChild(seats);

        card.appendChild(status);

        card.appendChild(online);

        card.appendChild(button);

        // Append card to container
        container.appendChild(card);

    }

    // Display total events
    const info = document.getElementById("json-info");

    if (info) {

        info.textContent = "Total Events: " + eventList.length;

    }

}


/* ================ B4: HAMBURGER MENU ================ */

if (menuButton && navLinks) {

    menuButton.addEventListener("click", function () {

        navLinks.classList.toggle("active");

        const expanded = navLinks.classList.contains("active");

        menuButton.setAttribute(
```

```
            "aria-expanded",
            String(expanded)
        );

        console.log("Menu toggled:", expanded);

    });

}


// Close menu after clicking navigation
navItems.forEach(function (link) {

    link.addEventListener("click", function () {

        navLinks?.classList.remove("active");

        menuButton?.setAttribute("aria-expanded", "false");

    });

});


/* ================ B5: EVENT OBJECT + EVENT.TARGET ================ */

// Event delegation and event bubbling
if (container) {

    container.addEventListener("click", function (event) {

        const clickedElement = event.target.closest(".rsvp-button");

        if (!clickedElement || !container.contains(clickedElement)) {
            return;
        }

        // Find clicked event card
        const card = clickedElement.closest(".event-card");

        const eventTitle = card.getAttribute("data-event-title");

        console.log("RSVP clicked:", eventTitle);

        clickedElement.textContent = "Selected";

        clickedElement.classList.toggle("selected");

        const info = document.getElementById("json-info");

        if (info) {

            info.textContent = "Selected Event: " + eventTitle;

        }

    });

}


/* ================ B6: SEARCH + ESCAPE ================ */

// Search the Featured Events section
const searchInput = searchBox;


// Find the section containing the search box
function getFeaturedCards() {

    if (!searchInput) {
        return [];
    }

    let searchSection = searchInput.closest("section");

    // Fallback if the search box is not inside a section
    if (!searchSection) {

        searchSection =
            searchInput.closest(".featured-events") ||
            searchInput.parentElement?.parentElement;

    }

    if (!searchSection) {
        return [];
    }
```

```
    // Find cards only inside Featured Events
    const cards = searchSection.querySelectorAll(
        ".event-card, .featured-card, .event, article"
    );

    // Do not include the search input or its parent
    return Array.from(cards).filter(card => {

        return !card.contains(searchInput);

    });

}



// Input event: filter featured event cards
if (searchInput) {

    searchInput.addEventListener("input", function () {

        const searchText = searchInput.value
            .trim()
            .toLowerCase();

        const featuredCards = getFeaturedCards();

        let visibleCount = 0;

        // Search through Featured Events cards
        featuredCards.forEach(function (card) {

            const cardText = card.textContent.toLowerCase();

            const matches = cardText.includes(searchText);

            // Show matching cards and hide others
            card.style.display = matches ? "" : "none";

            if (matches) {
                visibleCount++;
            }

        });

        console.log("Search:", searchText);

        console.log("Matching featured events:", visibleCount);

        // Show message when no events match
        let noResults = document.getElementById("no-search-results");

        if (!noResults && searchInput.parentElement) {

            noResults = document.createElement("p");

            noResults.id = "no-search-results";

            noResults.textContent = "No matching events found.";

            searchInput.parentElement.appendChild(noResults);

        }

        if (noResults) {

            noResults.style.display =
                visibleCount === 0 ? "block" : "none";

        }

    });



    // Escape key clears search
    searchInput.addEventListener("keydown", function (event) {

        if (event.key === "Escape") {

            searchInput.value = "";

            // Show all Featured Events again
            getFeaturedCards().forEach(function (card) {

                card.style.display = "";

            });

            const noResults = document.getElementById("no-search-results");

            if (noResults) {
                noResults.style.display = "none";
```

```
            }

            console.log("Search cleared");

        }

    });

}


/* ================ B7: LOAD JSON USING FETCH ================ */

async function loadJSON() {

    try {

        // Loading state
        if (container) {

            container.textContent = "Loading events...";

        }

        console.log("Loading events.json...");

        const response = await fetch("events.json");

        if (!response.ok) {

            throw new Error("JSON file not found");

        }

        const jsonText = await response.text();

        // Convert JSON text into JavaScript objects
        const loadedEvents = JSON.parse(jsonText);

        if (!Array.isArray(loadedEvents)) {

            throw new Error("JSON must contain an array");

        }

        // Check event data
        events = loadedEvents;

        totalEvents = events.length;

        // Success state
        console.log("JSON loaded successfully");

        console.log("Total Events:", totalEvents);

        // Render event cards
        renderEvents(events);

        console.log("First event:", events[0]?.title);

        console.log(
            "Seats of last event:",
            events[events.length - 1]?.seats
        );

        // Process all events
        processEvents();

        // Function return
        if (events.length > 0) {

            console.log(
                "Formatted event:",
                formatEvent(events[0])
            );

        }

        // Available events
        console.log("Available events:", availableEvents());

        // JSON round trip
        const jsonString = JSON.stringify(events, null, 2);

        console.log("JSON String:");

        console.log(jsonString);

        if (jsonOutput) {

            jsonOutput.textContent = jsonString;
```

```
        }

        // JSON file size
        const jsonSize = new Blob([jsonText]).size;

        window.jsonSize = jsonSize;

        checkFileSize();

    }
catch (error) {

    console.error("JSON Error:", error);

    if (container) {

        container.classList.add("fetch-error");

        container.textContent =
            "Sorry! Events could not be loaded. Please try again.";

    }

}
}



/* ================ B7: LOAD XML USING FETCH ================ */

async function loadXML() {

    try {

        const response = await fetch("events.xml");

        if (!response.ok) {

            throw new Error("XML file not found");

        }

        const xmlText = await response.text();

        // Parse XML
        const parser = new DOMParser();

        const xmlDoc = parser.parseFromString(
            xmlText,
            "application/xml"
        );

        // Check XML parsing errors
        if (xmlDoc.querySelector("parsererror")) {

            throw new Error("Invalid XML file");

        }

        // Read first event title
        const titles = xmlDoc.getElementsByTagName("title");

        if (titles.length > 0 && xmlTitle) {

            xmlTitle.textContent = titles[0].textContent;

        }

        // XML file size
        window.xmlSize = new Blob([xmlText]).size;

        checkFileSize();

        console.log("XML loaded successfully");

    }

    catch (error) {

        console.error("XML Error:", error);

        if (xmlTitle) {

            xmlTitle.textContent = "XML could not be loaded";

        }

    }

}
```

```
/* ================ B5: FILE SIZE COMPARISON ================ */

function checkFileSize() {

    if (
        window.jsonSize !== undefined &&
        window.xmlSize !== undefined &&
        fileSize
    ) {

        fileSize.textContent =
            `JSON Size: ${window.jsonSize} bytes | XML Size: ${window.xmlSize} bytes`;

    }

}



/* ================ B8: FORM VALIDATION ================ */

// Custom validation rule
if (studentName) {

    studentName.setAttribute(
        "pattern",
        "[A-Za-z ]{3,}"
    );

    studentName.setAttribute(
        "title",
        "Enter at least 3 letters"
    );

    studentName.addEventListener("input", function () {

        if (studentName.value.trim().length < 3) {

            studentName.setCustomValidity(
                "Name must contain at least 3 characters"
            );

        }

        else {

            studentName.setCustomValidity("");

        }

    });

}



// Validate registration form
if (eventForm) {

    eventForm.addEventListener("submit", function (event) {

        if (!eventForm.checkValidity()) {

            event.preventDefault();

            eventForm.reportValidity();

            console.log("Form validation failed");

        }

        else {

            event.preventDefault();

            console.log("Registration successful");

            alert("Registration submitted successfully!");

        }

    });

}



/* ================ B9: DEVTOOLS TESTING ================ */

console.log("================================");

console.log(boardTitle);
```

```
console.log("Online:", isOnline);

console.log("Default Seats:", defaultSeats);

console.log("Assignment 6 + 7 JavaScript loaded");

console.log("================================");


/* ================ RUN FUNCTIONS ================ */

// Load JSON and XML
loadJSON();

loadXML();
```